

Aulas 3 e 4

- Transferência de informação entre os periféricos e a memória
 - E/S por interrupção (*interrupt driven I/O*)
 - As interrupções no ciclo de instrução do CPU
 - Processamento de interrupções
 - Organizações alternativas do sistema de interrupções
 - E/S por acesso direto à memória (DMA)
 - Princípio de funcionamento
 - Modos de funcionamento
 - Configurações
- se tivermos ação a fazer daqui a n horas de vez, em quando olho para o relógio ou método alarme*

José Luís Azevedo, Bernardo Cunha, Tomás O. Silva, P. Bartolomeu

E/S por interrupção

sinalização do evento é feita por hardware

- Na técnica de E/S por interrupção quando o periférico está pronto para trocar dados sinaliza o CPU

evento de hardware gerado por um periférico

- Uma interrupção, depois de reconhecida, faz com que o CPU suspenda temporariamente a execução do programa em curso para executar a rotina de tratamento correspondente

- A rotina associada à interrupção designa-se por **rotina de serviço à interrupção** (RSI) ou ***interrupt handler***

desencadeia a ação que dá seguimento à interrupção
não aparece no código (é feito automaticamente por hardware)

- A transferência é feita (tal como na E/S programada) pelo CPU mas o tempo de espera é eliminado, uma vez que a interrupção ocorre quando o periférico está pronto para a troca de dados
- Esta técnica mascara o problema da longa latência descrito no exemplo de leitura de informação de uma unidade de disco (aula 2)

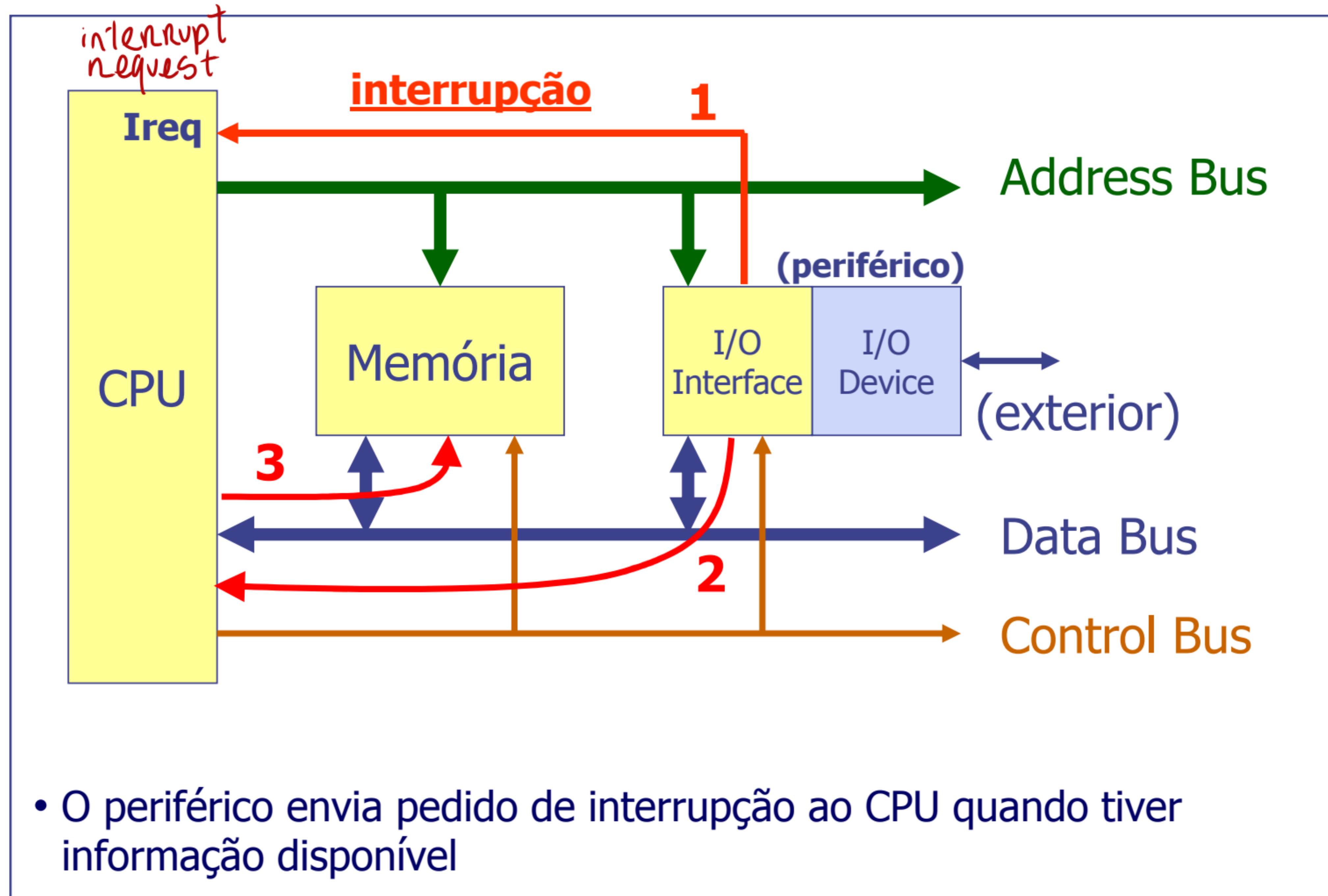
E/S por interrupção

também se passa na escrita

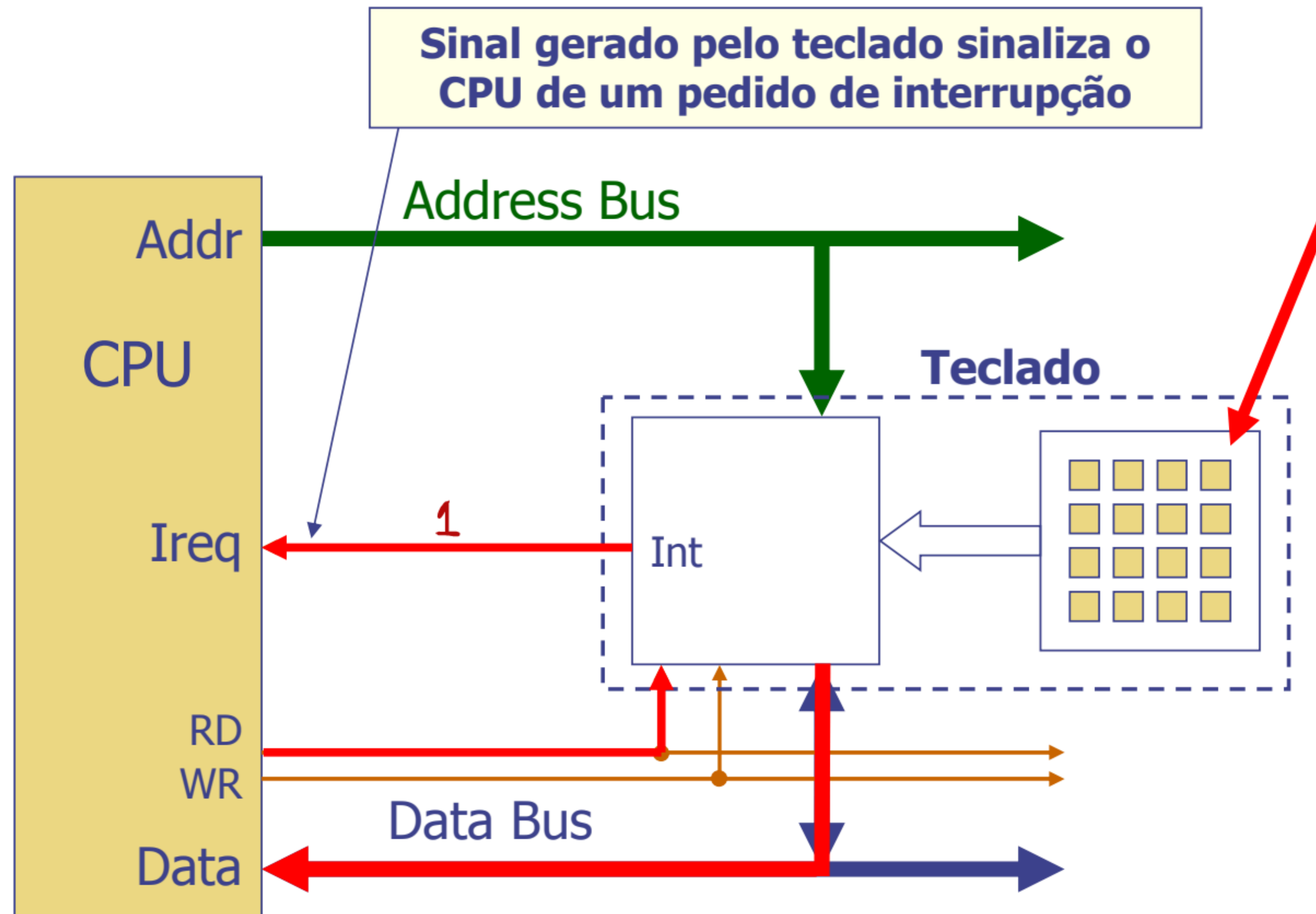
Exemplo: leitura de dados de um periférico

- CPU envia pedido de informação ao periférico (escrita num registo de controlo do periférico)
- CPU continua a execução do programa (com outras tarefas)
- Quando ^{o periférico} tiver informação disponível, o periférico gera um pedido de interrupção ao CPU
_{↑ ligação directa ao CPU}
- CPU atende a interrupção:
 - Suspende a execução do programa corrente
 - Salta para a **rotina de atendimento à interrupção** (*interrupt handler*) que transfere a informação _{↳ tipo jal}
 - Retoma a execução do programa suspenso

E/S por interrupção



E/S por interrupção (exemplo)



- Ireq (*Interrupt request*): entrada de interrupção do CPU

E/S por interrupção (exemplo de leitura)

tem de
ser uma
variável
global

```
// Configure I/O device and interrupt system
```

```
(...)
```

```
bytesReceived = 0
```

```
While(1) {
```

```
    (...) // Do other tasks/process
```

```
    (...) // data
```

```
}
```

sempre

```
void interrupt isr(void)
```

```
{
```

```
    Read byte from I/O Module
```

```
    Write byte into Memory
```

```
    bytesReceived++
```

```
}
```

*n é chamada no programa
ent n dá pa lhe
passar parametros*

- **Não existe qualquer ciclo de espera.** O periférico gera o pedido de interrupção quando está pronto a transferir a informação
- O programa em execução **pode ser interrompido a qualquer momento**
- A Rotina de Serviço à Interrupção (RSI) tem que **salvaguardar o contexto** do programa (registos internos, ...) antes de executar qualquer ação. O contexto salvaguardado tem que ser repostado antes de se terminar a RSI
- A palavra-chave "**interrupt**" distingue uma função do tipo RSI de uma função normal

, diz ao compilador para salvaguardar os registos

*registos
CPU que
vai usar*

pode ser chamada pelo hardware em qualquer ponto do programa

E/S por interrupção

- **Exemplo:** versão *interrupt-driven* do exemplo de comutação do estado de um LED (RD3) a cada transição de 0 para 1 de um sinal de entrada

```
main:  # config PIC32 ports and interrupt system
      (...)
while: (...)  # CPU executa outras tarefas
      instr.1
      instr.2
      (...)
      instr.n
      j while

      isr: # save program context
           (...) # prólogo
           lui $t0, ADDR_BASE_HI
           lw  $t1, LATD($t0) #
           xori $t1, $t1, 0x0008 #
           sw  $t1, LATD($t0) # RD3=!RD3;
           # restore program context
           (...) # epílogo
           eret # exception return
```

Transição de 0 para 1 em RD0 inicia uma interrupção

- Não existe qualquer ciclo de espera
- O programa em execução é interrompido quando é detetada uma transição 0 para 1 na entrada de interrupção do CPU
- Quando acaba a execução do *Interrupt Handler* (rotina "isr"), o CPU retoma a execução do programa interrompido

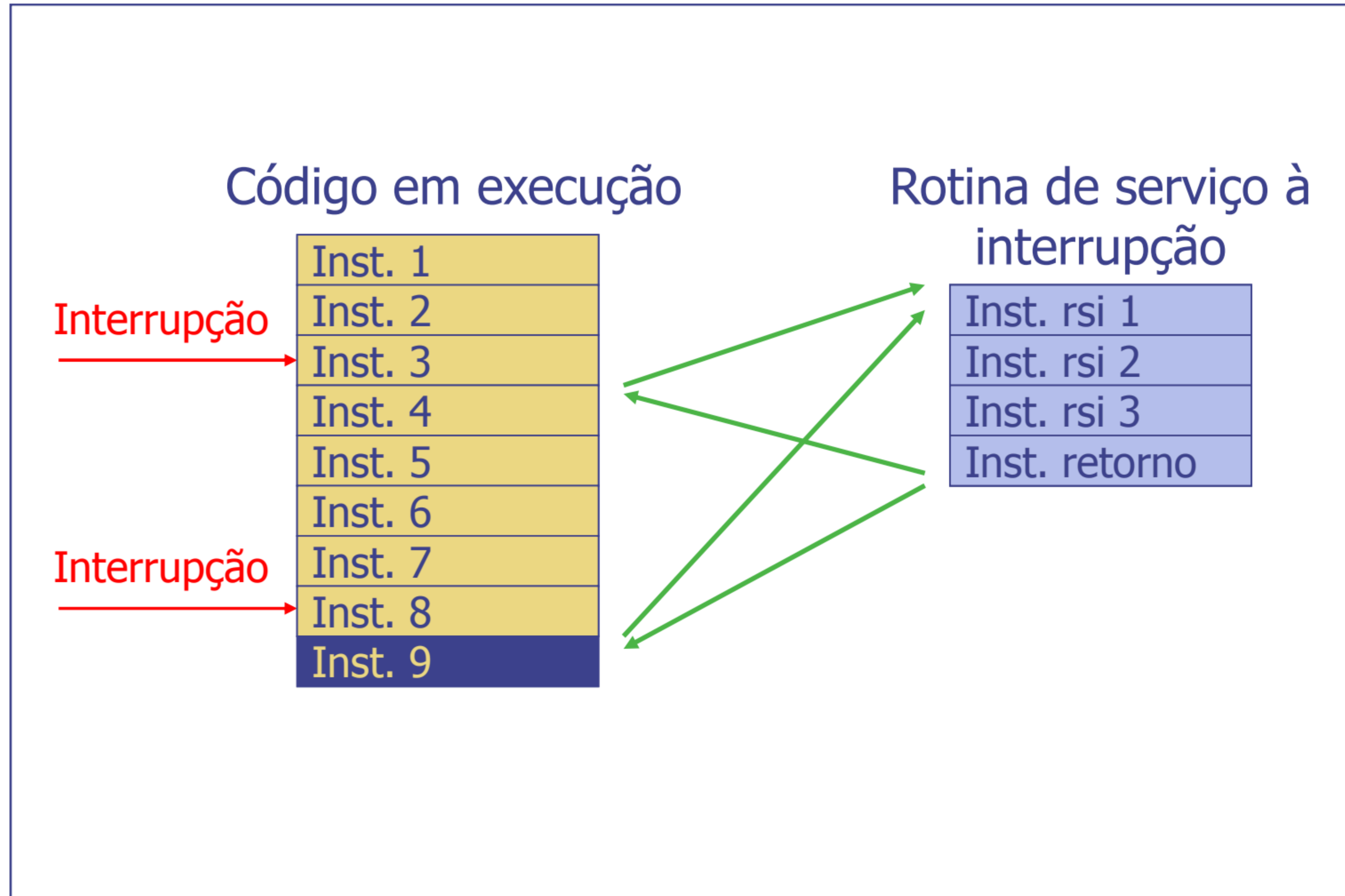
jr \$ra mas para rotinas isr

Exceções e interrupções

são tratadas de forma semelhante

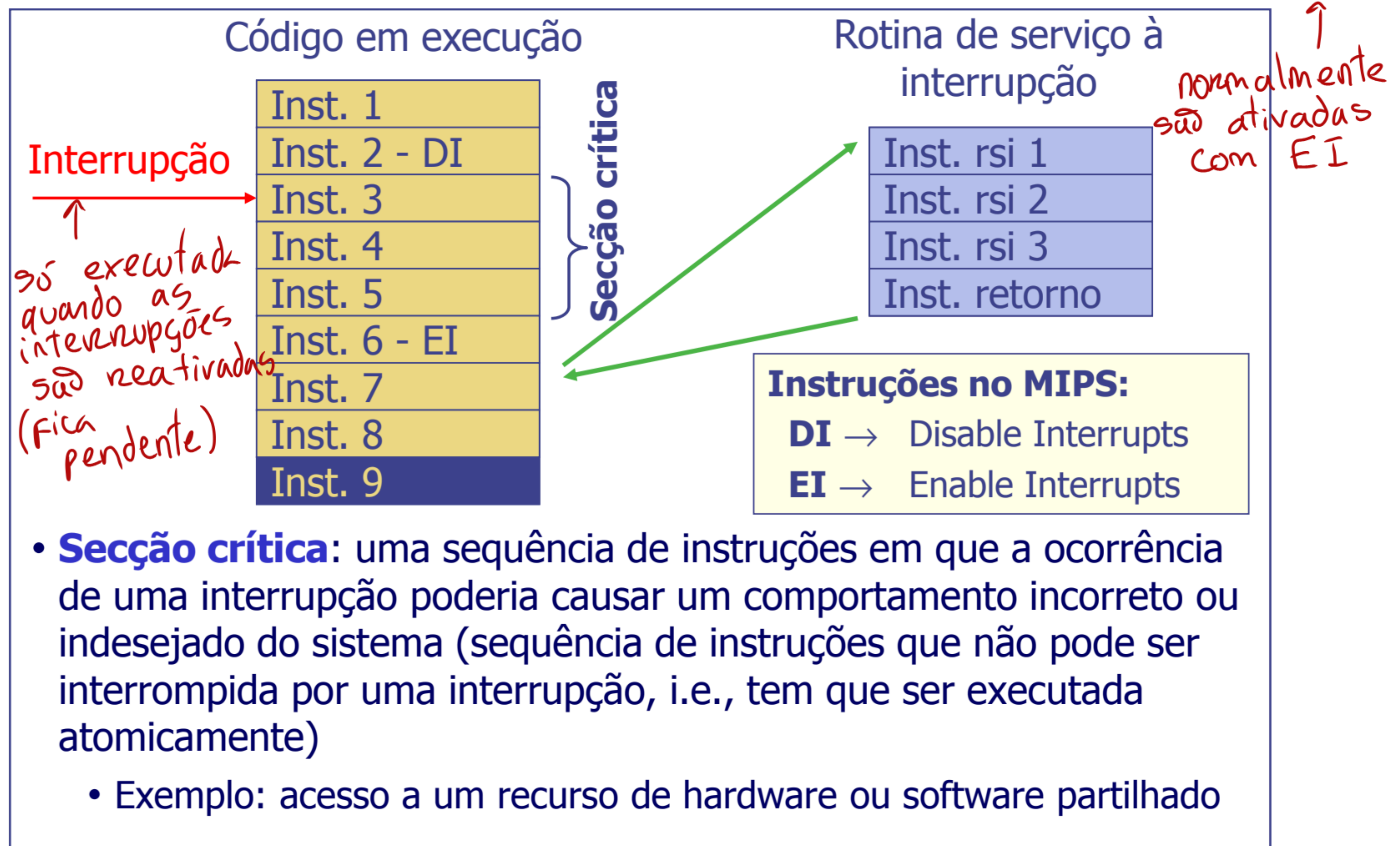
- Exceções e interrupções são eventos que, não sendo *branches* ou *jumps*, alteram o fluxo normal de execução do programa; existem duas fontes distintas de eventos deste tipo:
 - Eventos com origem no CPU, inesperados e decorrentes da execução das próprias instruções – **exceções** – o programa aborta sempre
 - exemplos: divisão por zero, *overflow*, acesso a endereço inválido
 - Eventos com origem externa ao CPU ^{em periféricos} que surgem assincronamente com o funcionamento deste – **interrupções**. Exemplo: quando é premida uma tecla do teclado do exemplo anterior
- **Exceções**: a instrução que gera a exceção não termina: a execução é passada de imediato para a rotina de tratamento da exceção
 - a instrução interrompida acaba por executar até ao fim
- **Interrupções**: a unidade de controlo apenas verifica se há algum pedido de interrupção pendente antes de iniciar o *fetch* de uma nova instrução
- Processamento de interrupções e exceções é semelhante

Processamento de interrupções



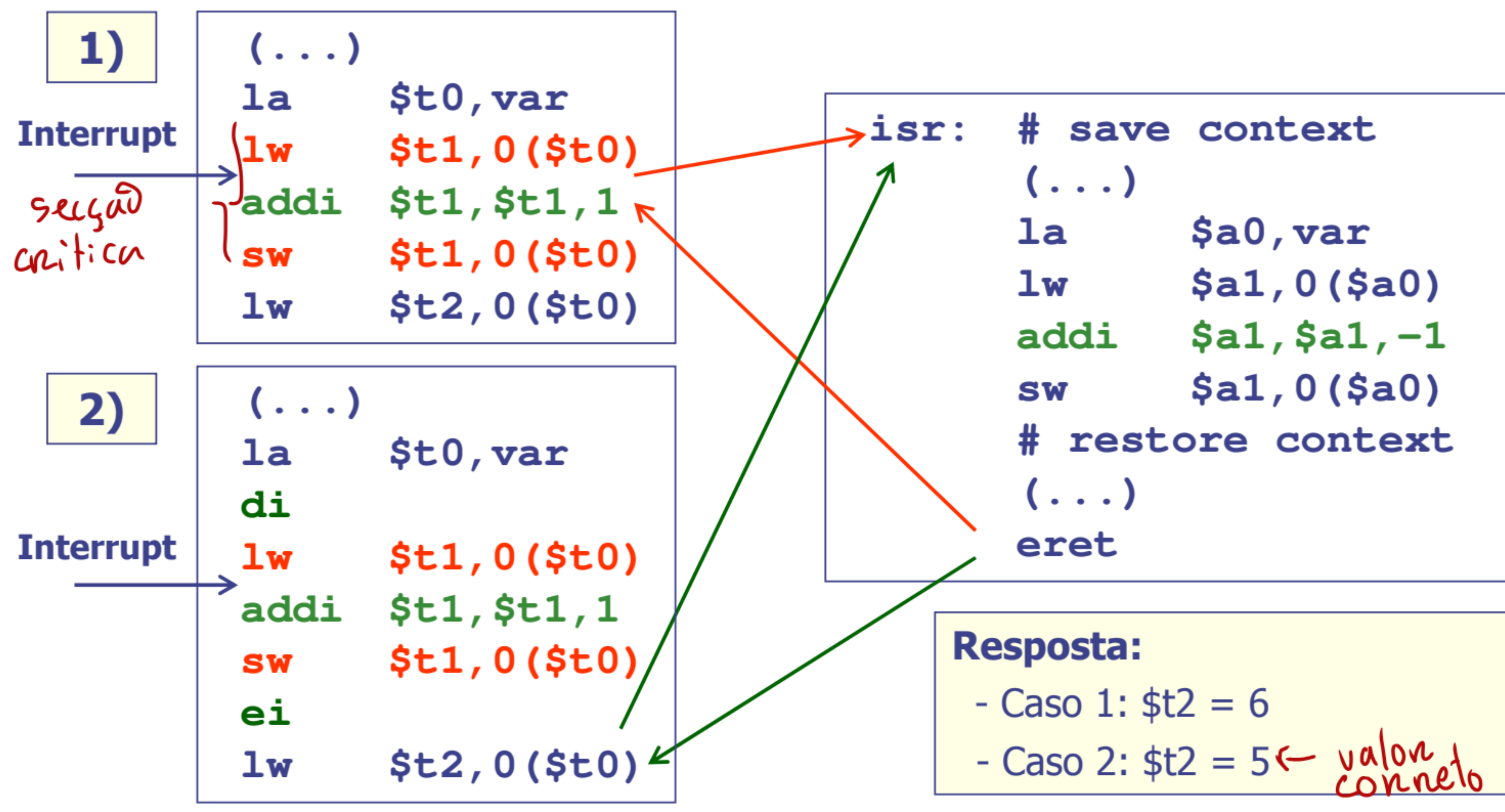
Ativação/desativação global das interrupções

por defeito,
quando o programa arranca,
as interrupções
estão desativadas



Processamento de interrupções – secção crítica (exemplo)

- A variável "**var**" pode ser lida e alterada na RSI e no programa principal. Se "**var**" tem o valor 5 antes da ocorrência da interrupção, qual o valor lido para o registo **\$t2**, no caso 1 e no caso 2?



Processamento de interrupções pelo CPU

- Em termos gerais, o processamento de uma interrupção é efetuado, pelo CPU, nos seguintes passos:

1. Identificação da fonte de interrupção (nos casos em que tal é efetuado por hardware) e obtenção do endereço da RSI
 2. Salvaguarda do contexto atual do CPU (valor corrente do PC e de flags de estado associadas ao sistema)
↳ Cpu tem que identificar quem gerou a interrupção
endereço de retorno
 3. Desativação das interrupções
↳ na arquitetura intel p.ex
 4. Carregamento do PC com o endereço da RSI (PC ← Endereço da RSI, i.e., salto para a 1ª instrução da RSI)
↳ n é o caso do MIPS mas é a norma
endereço obtido no ponto 1
 5. Execução da RSI até encontrar a instrução de retorno
 6. Execução da instrução de retorno da RSI (e.g. `eret`, no MIPS)
- Reposição do contexto salvaguardado (PC e flags) e reativação das interrupções => regresso ao programa interrompido, com a execução da instrução que teria sido executada se não tivesse acontecido a interrupção

quando
executado
"eret"

Processamento de interrupções pela RSI

- Ações gerais que devem ser implementadas na Rotina de Serviço à Interrupção (software):

1. Salvaguarda do contexto do programa que foi interrompido:
registos internos do CPU → memória (*stack*) ("**PRÓLOGO**")

→ código
gerado pelo
compilador

2. .. ações associadas ao processamento da interrupção..

3. Reposição do contexto do programa interrompido:
registos internos do CPU ← memória (*stack*) ("**EPÍLOGO**")

4. Conclusão da RSI com a instrução de retorno (do tipo "Return From Interrupt / exception" – "**eret**" no caso do MIPS)

Overhead do método de transferência por interrupção

- **Latência da interrupção:** define-se como o tempo que decorre desde a ocorrência do evento que desencadeia a interrupção até à execução da primeira instrução da Rotina de Serviço à Interrupção (pontos 1 a 4 do slide 12 mais eventual conclusão da secção crítica do código) *tempo desde o evento de interrupção até ao início da RSI*
- *custos de relógio gastos no prólogo e epílogo*
- O **overhead** global do método de transferência por interrupção é, no essencial, causado pela mudança de contexto:
 - A rotina de serviço à interrupção tem que, à entrada, **salvaguardar o contexto do programa interrompido**
 - Antes de abandonar a RSI tem que **repor o contexto salvaguardado**
 - A título de exemplo, estas 2 operações requerem, no MIPS do PIC32, cerca de 50 instruções *(25 prólogo e 25 epílogo)*
(pode chegar às 80

Organização do sistema de interrupções

Vários periféricos
a gerar interrupções

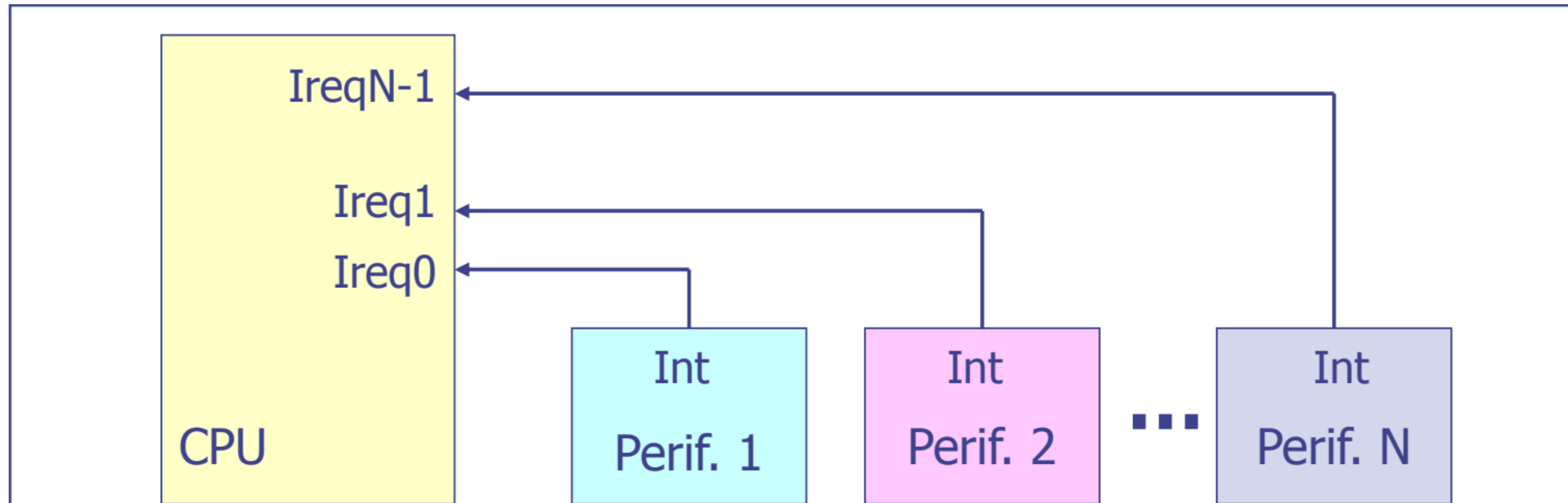
- Num sistema real é comum que vários periféricos possam gerar interrupções. Como organizar e priorizar esses pedidos?
- Organização do sistema de interrupções
 - **Múltiplas linhas de interrupção**
 - Cada periférico tem a sua linha dedicada
 - **Identificação da fonte de interrupção por software**
 - O programa verifica qual o periférico que gerou a interrupção
 - **Interrupções vetorizadas**
 - A identificação do periférico que gerou a interrupção é feita por hardware
- Gestão de prioridade
 - Como tratar pedidos simultâneos de interrupção?
 - Como definir níveis de prioridade entre diferentes fontes?

→ n entradas de interrupção (1 por periférico)

→ 1 linha a funcionar como "or" entre todos os periféricos
↳ por software

Opção 1:

Múltiplas linhas de interrupção



- Identificação automática da fonte de interrupção
- Uma RSI para cada fonte de interrupção
- Número máximo de dispositivos que podem gerar interrupção é igual ao número de linhas de interrupção do CPU - nº de entradas de interrupção que o CPU tem disponível
- Cada linha tem atribuída uma prioridade fixa (na implementação hardware pode ser usado um priority encoder)
 - No caso de haver 2 ou mais linhas de interrupção ativas simultaneamente, o CPU atende em 1º lugar a de mais alta prioridade

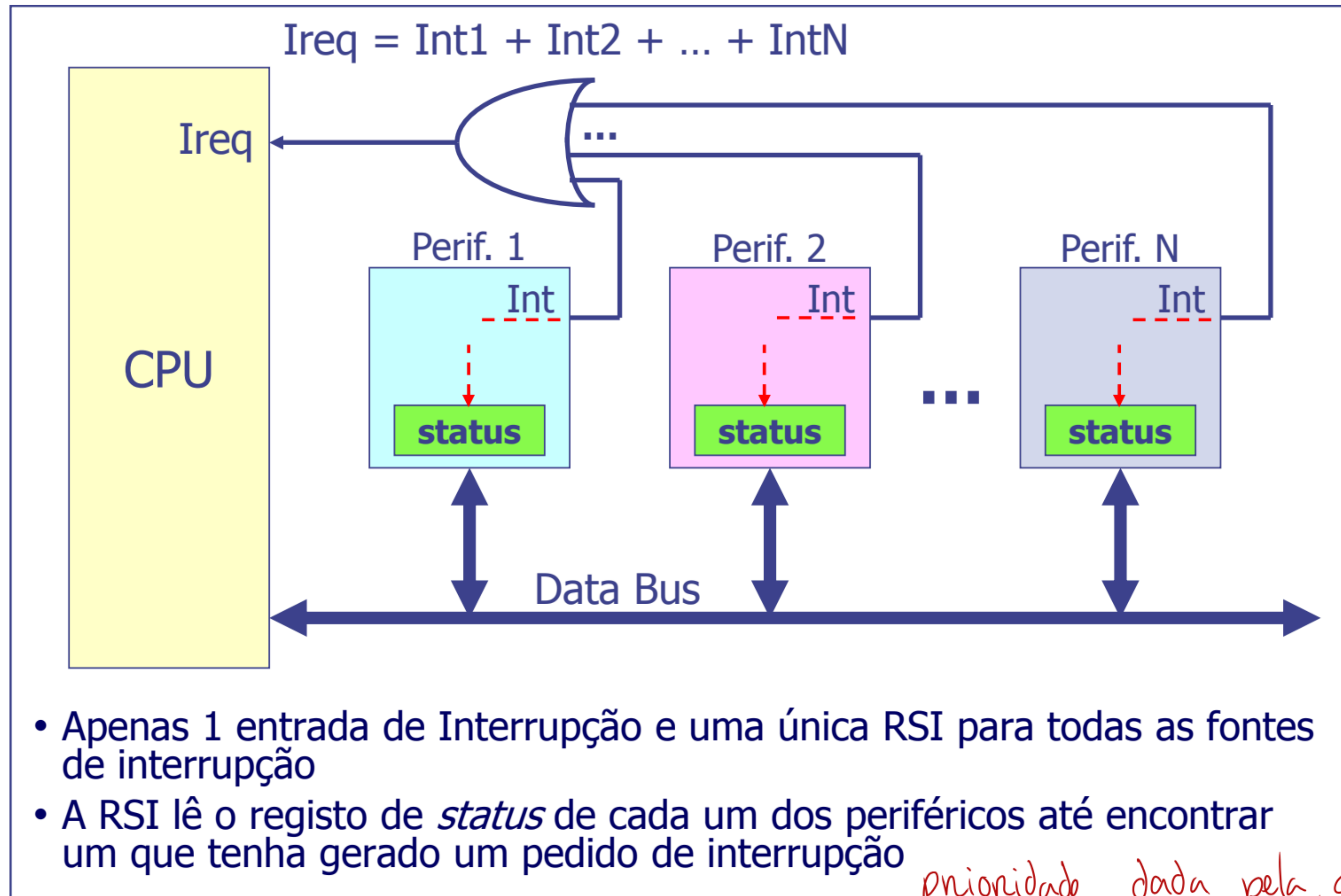
na PIC32 é programável

8 entradas e 3 saídas
se é a 7ª entrada ativa → saída = 111

a entrada mais elevada
(de maior peso)

Opção 2:

Identificação da fonte de interrupção por *software*



Identificação da fonte de interrupção por software

n usado pelo PIC32

- Exemplo de organização da Rotina de Serviço à Interrupção

```
void interrupt general_isr(void)
{
    Read status register of peripheral 1
    If( interrupt_bit = ON) {
        peripheral_isr_1()
    }

    Read status register of peripheral 2
    If( interrupt_bit = ON) {
        peripheral_isr_2()
    }

    (...)

    Read status register of peripheral n
    If( interrupt_bit = ON) {
        peripheral_isr_n()
    }
}
```

*dá pa mudar a ordem
o que equivale á prioridade*

Funções específicas para
tratamento dos pedidos de
interrupção de cada fonte

No caso de pedidos de
interrupção simultâneos, a
ordem pela qual os periféricos
são "questionados" determina a
prioridade no atendimento

• Mais simples

• Menos custos de hardware

Opção 3:

Interrupções vetorizadas

interrupção → CPU vê o vetor → CPU salta para a RSI correspondente

- CPU tem apenas 1 entrada de interrupção
- A identificação da fonte é feita por hardware
- Cada periférico possui um identificador único, designado por **vetor**
- Uma RSI para cada vetor de interrupção
- Durante o processo de atendimento, na fase de identificação da fonte, o periférico gerador da interrupção identifica-se através do seu vetor
- O vetor vai depois ser usado como índice de uma tabela que contém:
 - o endereço de cada uma das RSI, **ou**
 - instruções de salto incondicional para as RSI

usado por Intel

↳ preenchida pelo compilador

caso do MIPS

Interrupções vetorizadas

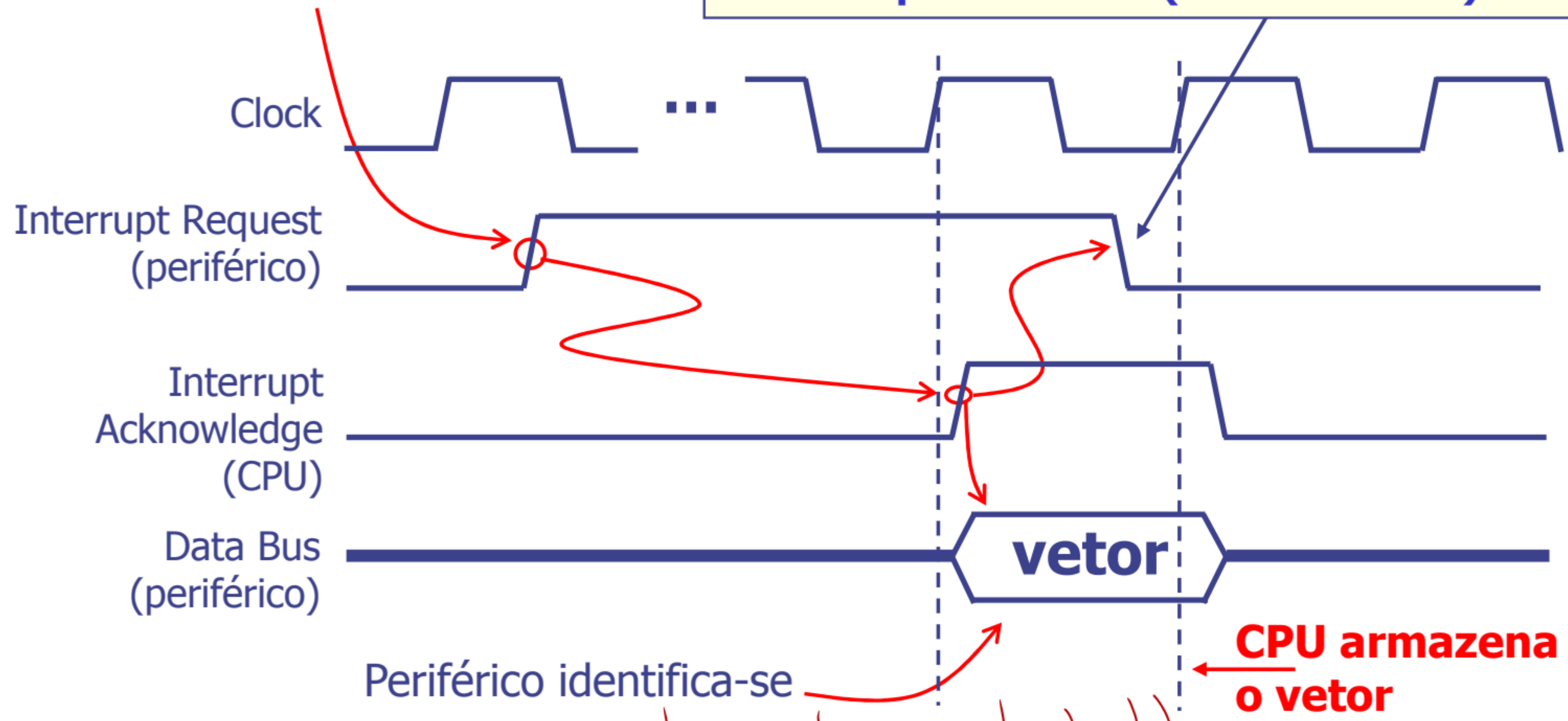
- A identificação da fonte de interrupção é feita por hardware num processo genericamente designado por "Interrupt acknowledge cycle"

Periférico gera pedido de interrupção

O reset do sinal que levou à ativação do Ireq pode, em algumas arquiteturas, ser feito por *software* (caso do PIC32)

→ n é automático

cpu tem saída extra



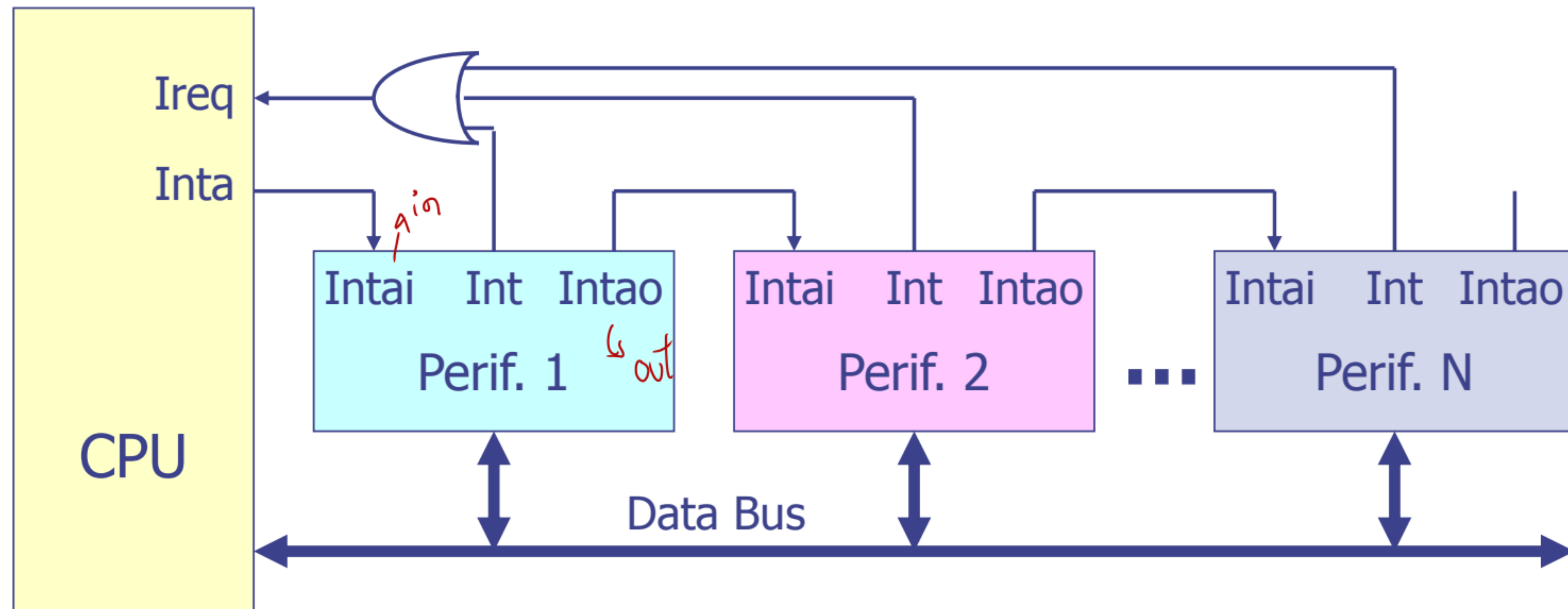
Periférico identifica-se e coloca o vetor no barramento de dados

CPU armazena o vetor

organização dos periféricos

Interrupções vetorizadas – *daisy chain*

- Periféricos podem estar organizados numa estrutura *daisy chain*



$$Ireq = Int1 + Int2 + \dots + IntN$$

se n foi o Perif. 1 a gerar a interrupção então ele propaga o **Intai** para o **Intao** até encontrar o Perif. que fez o pedido que dps manda para o Data Bus o vetor

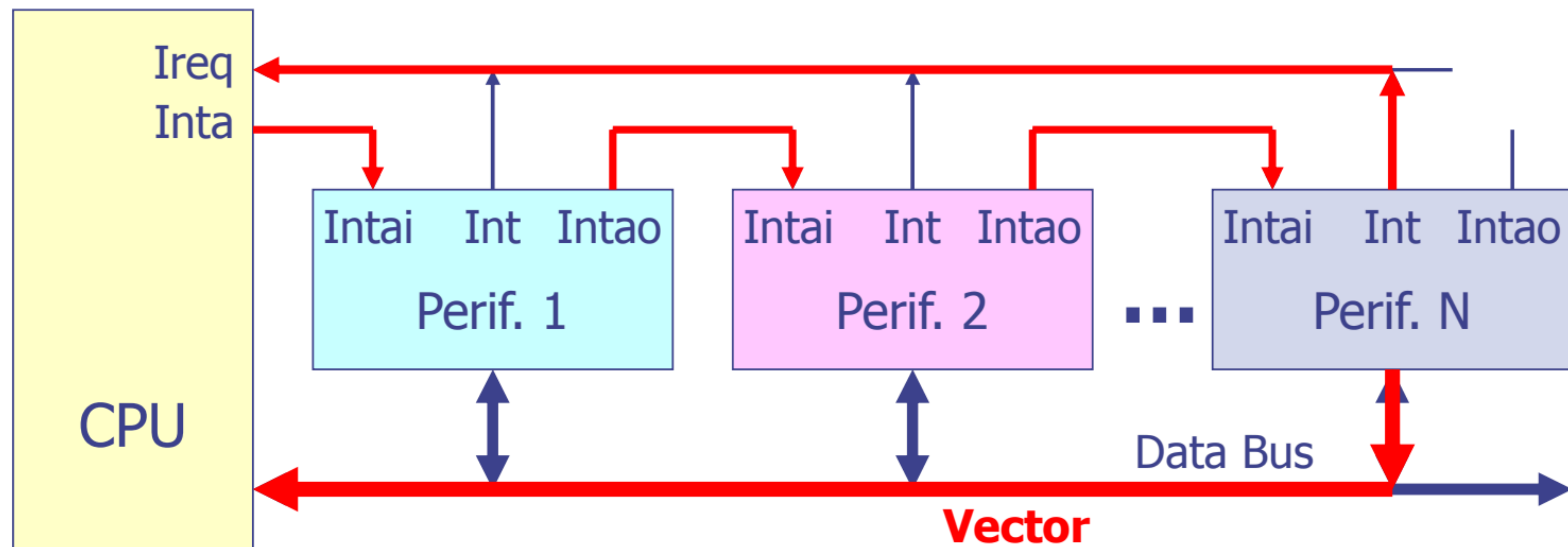
Intai/o - Interrupt Acknowledge in/out
ordem dos periféricos = prioridade

Interrupções vetorizadas – *daisy chain*

- Genericamente, o procedimento de identificação da fonte de interrupção num esquema de interrupções vetorizadas em que os periféricos estão organizados numa cadeia *daisy chain* é o seguinte:
 1. Quando o CPU deteta o pedido de interrupção ("Ireq") e está em condições de o atender ativa o sinal "Interrupt Acknowledge" ("Inta")
 2. O sinal "Inta" percorre a cadeia até ao periférico que gerou a interrupção
 3. O periférico que gerou a interrupção coloca o seu identificador (vetor) no barramento de dados e bloqueia a propagação do sinal "Interrupt Acknowledge"
 4. O CPU lê o vetor e usa-o como índice da tabela que contém os endereços das RSI ou instruções de salto para as RSI

Interrupções vetorizadas – *daisy chain*

- Exemplo de identificação da fonte de interrupção

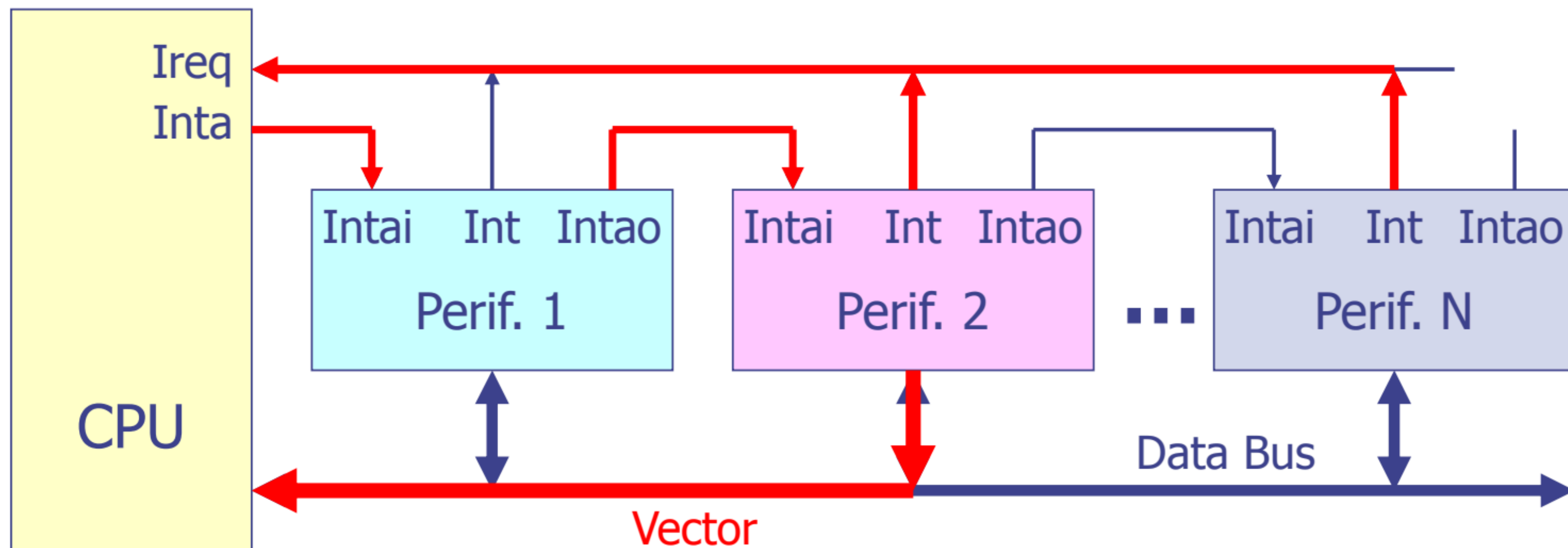


$$Ireq = Int1 + Int2 + \dots + IntN$$

Intai/Intao - Interrupt Acknowledge in/out

Interrupções vetorizadas – *daisy chain*

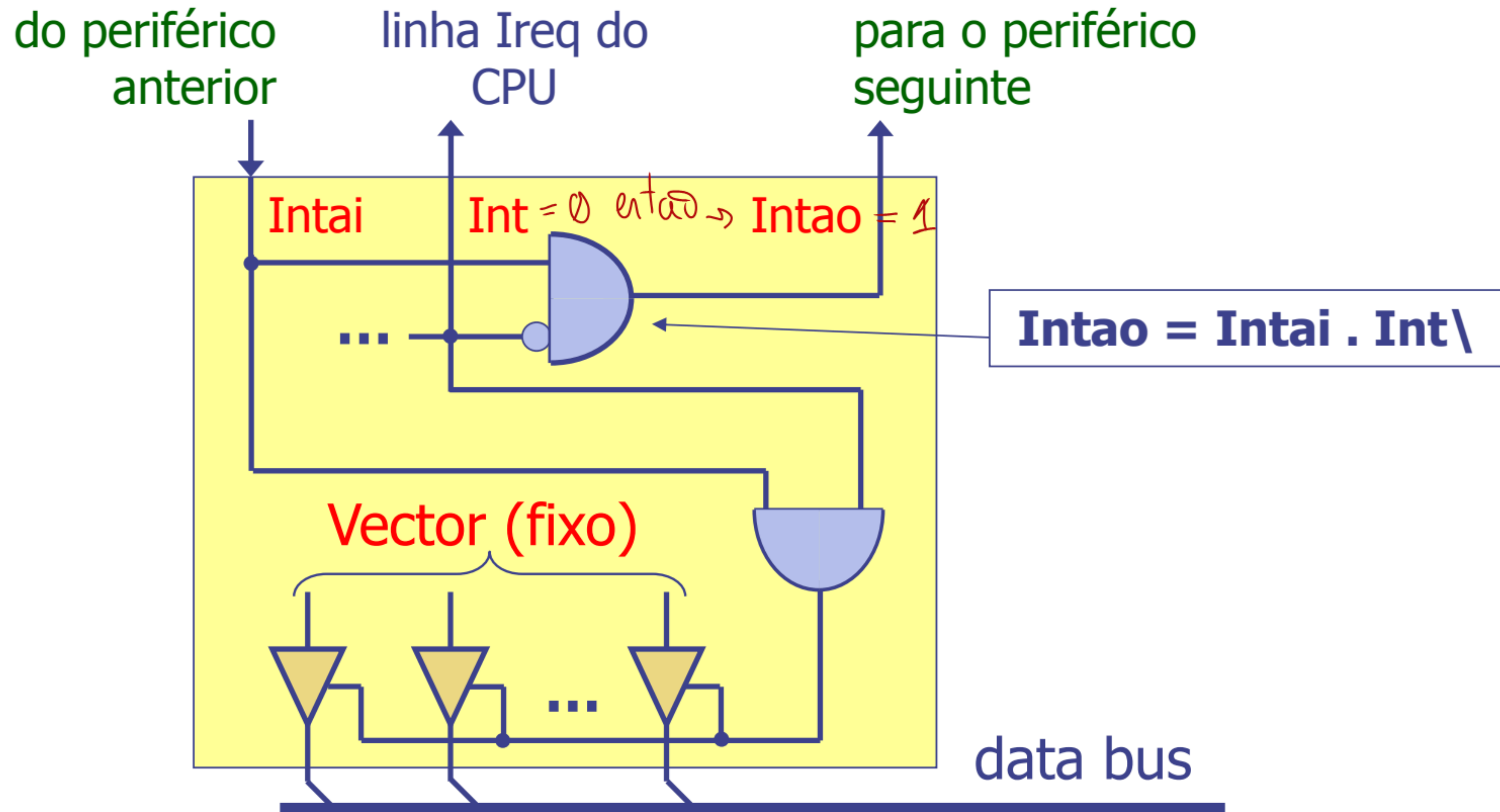
- Exemplo de identificação da fonte de interrupção, no caso em que dois periféricos têm a linha de interrupção ativa



- A ordem de colocação dos periféricos na cadeia, relativamente ao CPU, determina a sua prioridade

Interrupções vetorizadas – *daisy chain*

- Exemplo de implementação (arbitragem e identificação)



Interrupções vetorizadas – tabela de vetores

- Há duas formas de organizar a tabela de interrupções que associa um dado vetor a uma RSI

1. A tabela é inicializada com os endereços de todas as RSI
2. A tabela é inicializada com instruções de salto para as RSI

, n é bem uma tabela, é um array em que cada índice tem o endereço correspondente

- **Tabela inicializada com os endereços de todas as RSI:**
 - Durante a fase de identificação da fonte, o CPU recebe do periférico o vetor de interrupção
 - Utiliza depois esse vetor como índice na tabela de vetores, obtendo assim o endereço da rotina de serviço correspondente
 - O valor lido da tabela (endereço da primeira instrução da RSI) é depois carregado no *Program Counter*
 - este modelo é usado, por exemplo, na arquitetura Intel x86

Interrupções vetorizadas – tabela de vetores

Metodo do MIPS

- **Tabela inicializada com instruções de salto para as RSI:**

- São colocadas na tabela de interrupções instruções de salto para as RSI (em vez dos seus endereços)
- No processamento da interrupção o CPU usa o vetor como *offset* para saltar (jump) para a posição da tabela onde está, em geral, uma instrução de salto incondicional para a RSI a executar
- Este modelo é o usado na arquitetura MIPS
- O exemplo seguinte ilustra esta forma de organização, para 3 vetores:

Vector 0	→	0x9D000200	j	0x9D001C00	→	Salto para a RSI situada no endereço 0x9D001C00
		0x9D000204	nop			
Vector 1	→	0x9D000220	j	0x9D001D08		
		0x9D000224	nop			
Vector 2	→	0x9D000240	j	0x9D0020C4	→	Salto para a RSI situada no endereço 0x9D0020C4
		0x9D000244	nop			

temos que dizer os vetores que temos a usar e o compilador preenche a tabela com endereços

entrada/saída
✓

- E/S por acesso direto à memória (DMA)
 - Princípio de funcionamento
 - Modos de funcionamento
 - Configurações

Transferência por Acesso Direto à Memória (DMA)

- O problema da (possível) longa latência apresentada pelos periféricos é adequadamente tratada pela técnica de E/S por interrupção
- Esta técnica não resolve, contudo, a questão da transferência a taxas elevadas, uma vez que o limite é sempre imposto pelo facto de o CPU ter que executar um programa para efetuar a transferência
- Por exemplo, transferir dados de um periférico para a memória implica, do ponto de vista temporal:
 - O tempo de latência de resposta à interrupção + tempo de execução do prólogo da RSI
 - O tempo para executar instruções de: leitura de uma *word* do módulo de E/S do periférico, de escrita dessa *word* no endereço destino da memória e de atualização dos endereços origem e destino
 - Para cada iteração, o tempo necessário para verificar se todos os dados foram já transferidos (envolve a leitura de um ou mais registos do módulo de E/S do periférico e a verificação dos bits de status necessários)

Transferência por Acesso Direto à Memória (DMA)

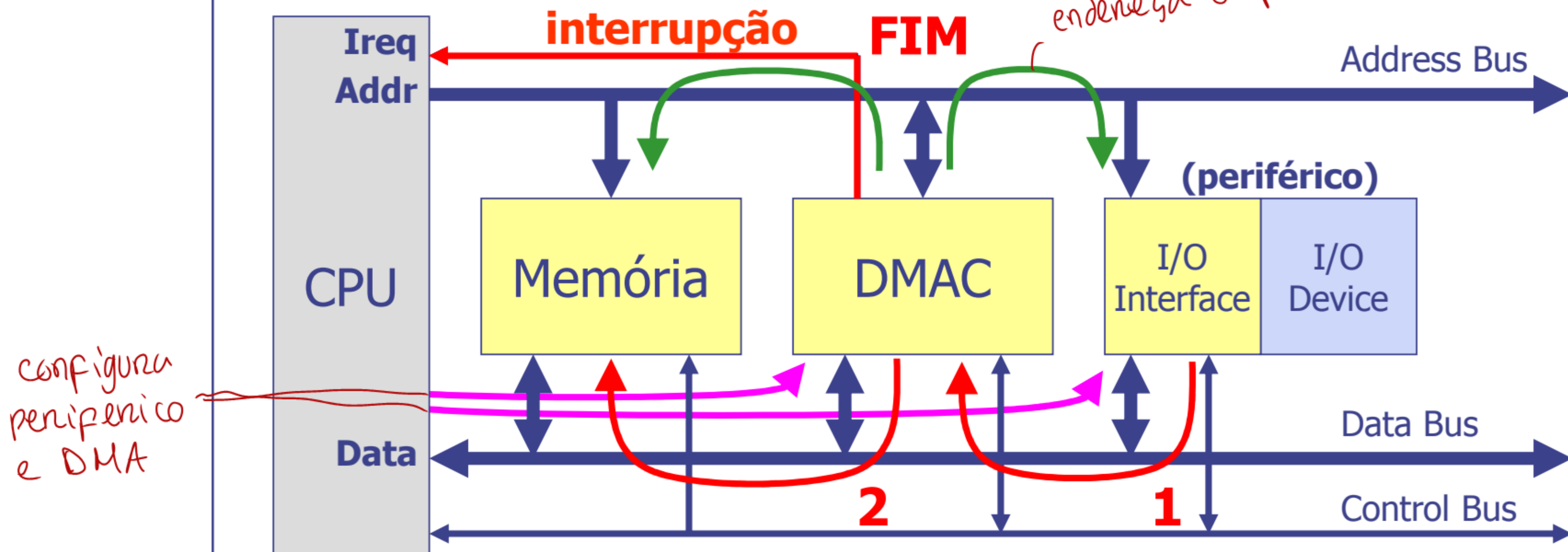
- Acresce ainda que, durante o período em que o CPU efetua a transferência, permanece totalmente ocupado, o que reduz a sua eficiência global na execução de outras tarefas
- A solução para mitigar o problema identificado é designada por **DMA** (*Direct Memory Access*) e consiste na transferência de informação do periférico diretamente para a memória (ou o contrário), sem intervenção do processador
- **Exercício 1:** um programa para transferir dados de 32 bits de um periférico para a memória é implementado com um ciclo com 10 instruções. Admitindo que o CPU funciona a 100 MHz e que o programa em causa apresenta um CPI de 2, determine a taxa de transferência máxima, em Bytes/s, ^{cycles per instr} que se consegue obter (suponha um barramento de dados de 32 bits) _{da pa transferir 32 bits de uma vez só}
- **Exercício 2:** admita que, de uma forma não realista, o programa do exercício anterior era constituído por apenas 2 instruções, e o CPI era 1. Qual seria nesse caso a taxa de transferência?

1 : Pa transferir 32 bits = 10 instr x 2 CPI = 20
 $\frac{100}{20} = 5$ Mega transferencia por segundo $\rightarrow$ 20 Mega bytes

2 : $\frac{100}{2} = 50$ Mega $\times 4$ bytes = 200 Megabytes

Transferência por Acesso Direto à Memória (DMA)

- Transferência de dados entre a memória e um periférico sem a intervenção do CPU. Um periférico especializado (DMAC – DMA Controller) efetua essa transferência



- Quando o controlador de DMA termina a transferência de todas as palavras, gera uma interrupção
- A transferência é **exclusivamente feita por hardware** pelo controlador de DMA, i.e., não é executado qualquer programa

Controlador de DMA (DMAC)

- Um controlador de DMA é um periférico que, do ponto de vista do modelo de programação, é semelhante a qualquer outro periférico
- Disponibiliza um conjunto de registros internos a que o CPU acede para configurar uma transferência de dados, nomeadamente:
 - Endereço origem da informação (endereço de leitura)
 - Endereço destino da informação (endereço de escrita)
 - Quantidade de informação a transferir (nº de bytes/words)
- Pode também ter registos com informação sobre o estado de uma transferência
- Do ponto de vista da operação realizada, o controlador de DMA é um periférico especializado na transferência de dados (cópia), de forma autónoma, em hardware, após programação feita pelo CPU
- Durante a transferência, o controlador de DMA tem a capacidade de controlar os barramentos de endereços, dados e controlo, como se fosse um CPU

mínimo
(pode ter um
registro status
a dizer o estado
da transferência)

Controlador de DMA (DMAC)

- Para efetuar uma transferência de um bloco de dados de **n** palavras, o controlador de **DMA** realiza, no essencial, ao nível dos barramentos, as mesmas operações que seriam realizadas por um programa executado pelo CPU: *nao é, necessariamente, feito sequencialmente*
 - **Lê** uma palavra (*byte* ou *word*) do dispositivo fonte (do **source address**) para um registo interno – "**fetch**"
 - **Escreve** a palavra guardada no registo interno no passo anterior, no dispositivo destino (no **destination address**) – "**deposit**"
 - Incrementa *source address* e *destination address*
 - Incrementa o **número de palavras transferidas**
 - Se não transferiu a totalidade do bloco, repete desde o início
- Para realizar estas tarefas o DMAC necessita de **controlar os barramentos de endereços** e de **dados** e ainda os sinais **rd** e **wr** *read e write*
- Note-se que a capacidade do DMAC para gerir os barramentos do sistema entra em conflito com capacidade idêntica do CPU

Controlador de DMA (DMAC)

Para gerir o conflito:

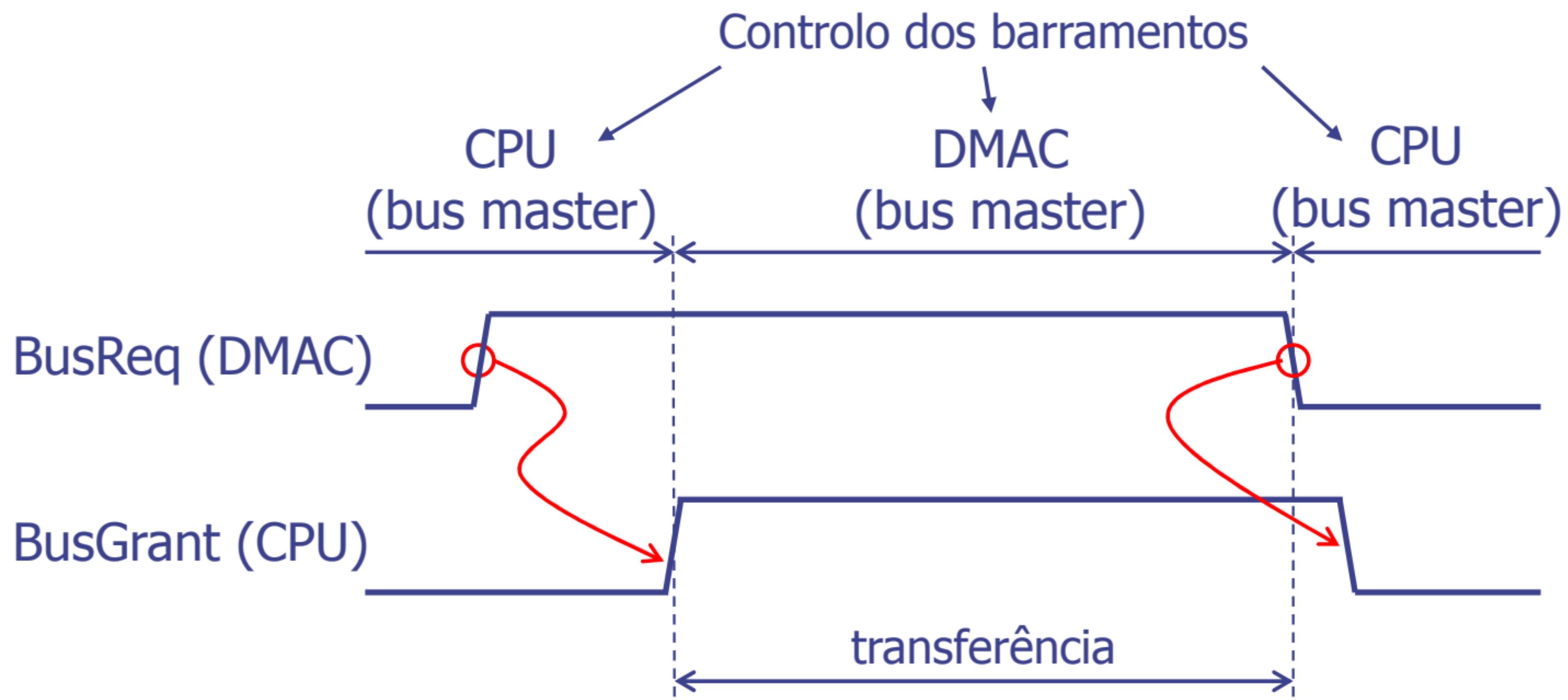
- Apenas um dos dois dispositivos, CPU e DMAC, pode estar ativo no barramento de cada vez... Por defeito é o CPU
- Isto é, só pode haver, em cada instante, um "**bus master**" (**só um dispositivo que é "bus master" pode controlar os barramentos**)
- Quando o DMAC necessita de aceder aos barramentos para realizar uma transferência, tem que primeiro ter autorização do CPU para ser o "bus master" *só enquanto realiza a transferência. nesse tempo o CPU n pode aceder à memória*
- O DMAC só inicia a transferência de informação quando tem a confirmação, por parte do CPU, de que é "bus master"
- O controlo dos barramentos por parte do DMAC é sempre temporário
- Quais são então os passos que o DMAC tem que seguir para fazer uma transferência de dados?

Controlador de DMA – passos para uma transferência

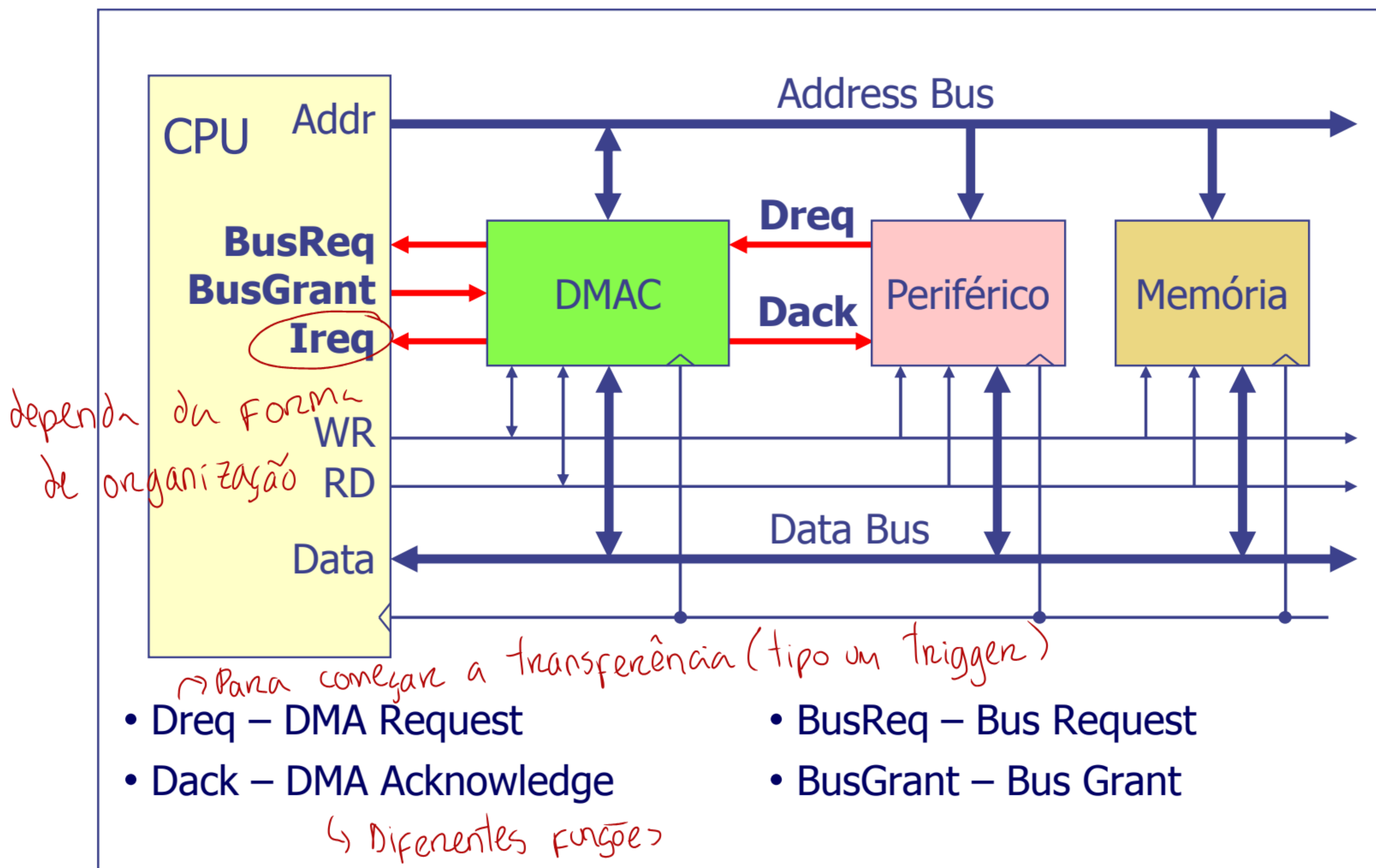
- DMAC pede ao CPU para ser *bus master* - ativa sinal "quero ser bus master"
- Espera até ter a confirmação do CPU
- Efetua a transferência:
 - Lê uma palavra (*byte* ou *word* de n bits) do dispositivo fonte (do *source address*) para um registo interno
 - Escreve a palavra guardada no registo interno, no passo anterior, no dispositivo destino (no *destination address*)
 - Incrementa *source address* e *destination address*
 - Incrementa o número de palavras transferidas
 - Se não transferiu a totalidade do bloco, repete
- Retira o pedido para ser *bus master* libertando, simultaneamente, os barramentos (ou seja, as suas ligações aos barramentos de dados, de endereços e de controlo passam a funcionar como entradas)
 - ↳ também gera interrupção "já acabei"
 - alta impedancia

Controlador de DMA

- Para se tornar um "bus master", o DMAC:
 1. ativa o sinal "**BusReq**" (*Bus Request*) e
 2. espera pela ativação do sinal "**BusGrant**" (CPU ativa *Bus Grant* quando está em condições de libertar os barramentos)



DMA – Hardware



DMA – Modos de operação

- **Bloco** – o DMAC assume o controlo dos barramentos até todos os dados terem sido transferidos → *Como visto anteriormente*
- **"Cycle Stealing"**
 - O DMAC assume o controlo dos barramentos durante 1 *bus cycle* e liberta-os de seguida ("rouba" 1 *bus-cycle* ao CPU) - transfere parcialmente 1 palavra (*fetch* ou *deposit*)
 - O CPU só liberta os barramentos nos ciclos em que não acede à memória (por exemplo, no estágio MEM de uma instrução aritmética na arquitetura MIPS, pipelined)
 - A transferência é mais lenta, mas o impacto do processo de DMA no desempenho do CPU é praticamente nulo - o DMAC aproveita os ciclos que não são, de qualquer modo, usados pelo CPU

Bus cycle (Ciclo de Bus) é a sequência fundamental de passos que é necessário realizar para executar uma única operação de transferência de dados (leitura ou escrita) entre dois componentes do sistema (por ex. CPU e a memória)

Um *bus cycle* é quantificado em ciclos do relógio de sincronização

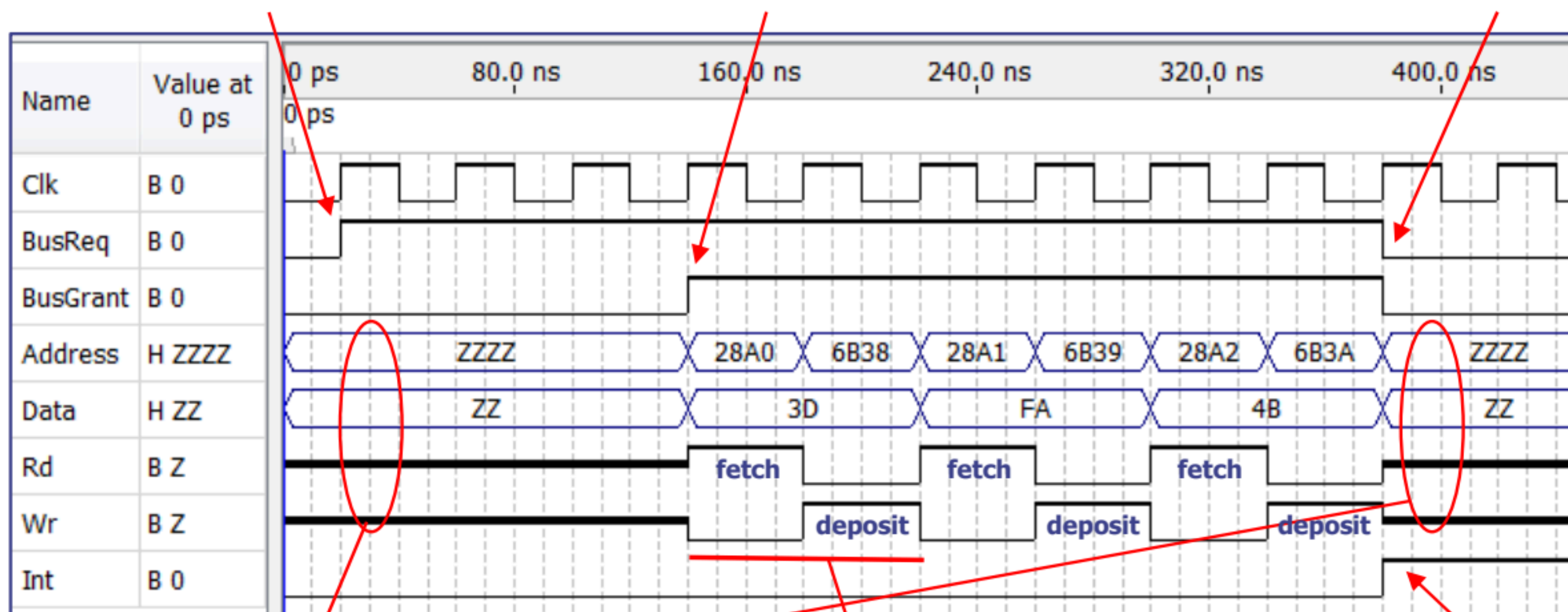
Modo Bloco (exemplo)

- Exemplo de uma transferência de 3 bytes (memória *byte-addressable*, barramento de dados de 8 bits, *bus cycle* = 1T):
 - Endereço de início na origem: 0x28A0, [0x28A0, 0x28A2]
 - Endereço de início no destino: 0x6B38, [0x6B38, 0x6B3A]

DMAC ativa BusReq

CPU responde com BusGrant

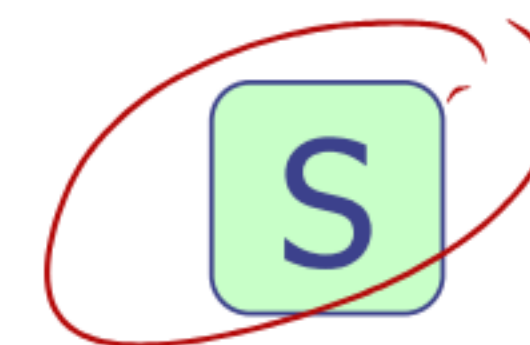
DMAC desativa BusReq



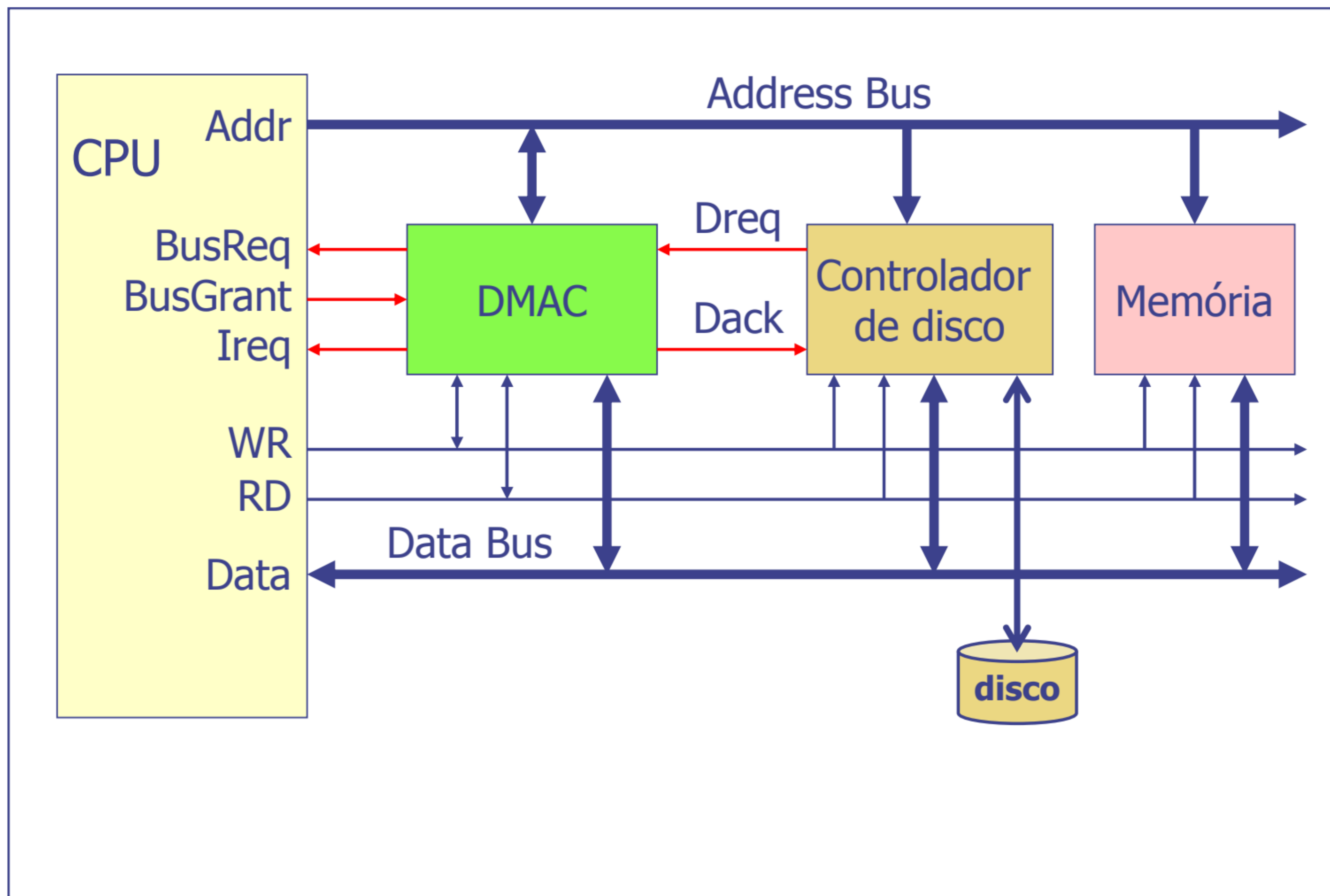
Barramentos do DMAC em alta impedância

Transferência do 1º byte

DMAC gera interrupção no fim da transferência



DMA – Hardware (exemplo)



Modo Bloco – Exemplo de uma transferência por DMA

1. O CPU envia um comando ao controlador do disco (DiskCtrl): leitura de um dado sector, número de palavras
2. O CPU programa o DMAC: endereço inicial da zona de dados a transferir (Controlador do disco), endereço inicial da zona destino (Memória), número de palavras a transferir
3. O CPU pode continuar com outras tarefas
4. Quando o DiskCtrl tiver lido a informação pedida para a sua zona de memória interna, ativa o sinal **Dreq** do DMAC (sinalizando dessa forma o DMAC de que a informação está pronta para ser transferida)
5. O DMAC ativa o sinal **BusReq**, pedindo autorização ao CPU para ser *bus master*, e fica à espera...
6. Logo que possa, o CPU coloca os seus barramentos em alta impedância e ativa o sinal **BusGrant** (o que significa que o DMAC passou a ser o *bus master*)
7. O DMAC ativa o sinal **Dack** e o DiskCtrl, em resposta, desativa o sinal **Dreq**

Modo Bloco – Exemplo de uma transferência por DMA

8. O DMAC efetua a transferência: i) lê do endereço-origem para um registo interno e ii) escreve do registo interno para o endereço-destino (incrementa endereços e número de palavras transferidas). Durante esta fase o CPU está impedido de aceder à memória de dados
9. Quando o DMAC termina a transferência:
 - Desativa o sinal **Dack**
 - Deixa de controlar os barramentos, isto é, os barramentos de dados, de endereços e de controlo passam a ser entradas
 - Desativa o sinal **BusReq**
 - Ativa o sinal de interrupção
10. O CPU quando deteta a desativação do BusReq desativa o sinal BusGrant e pode novamente usar os barramentos
11. Logo que possa, o CPU atende a interrupção gerada pelo DMAC

Modo "Cycle-Stealing" → Mais lento mas não interfere com o CPU (só dá bus grant quando n vai aceder a memória)

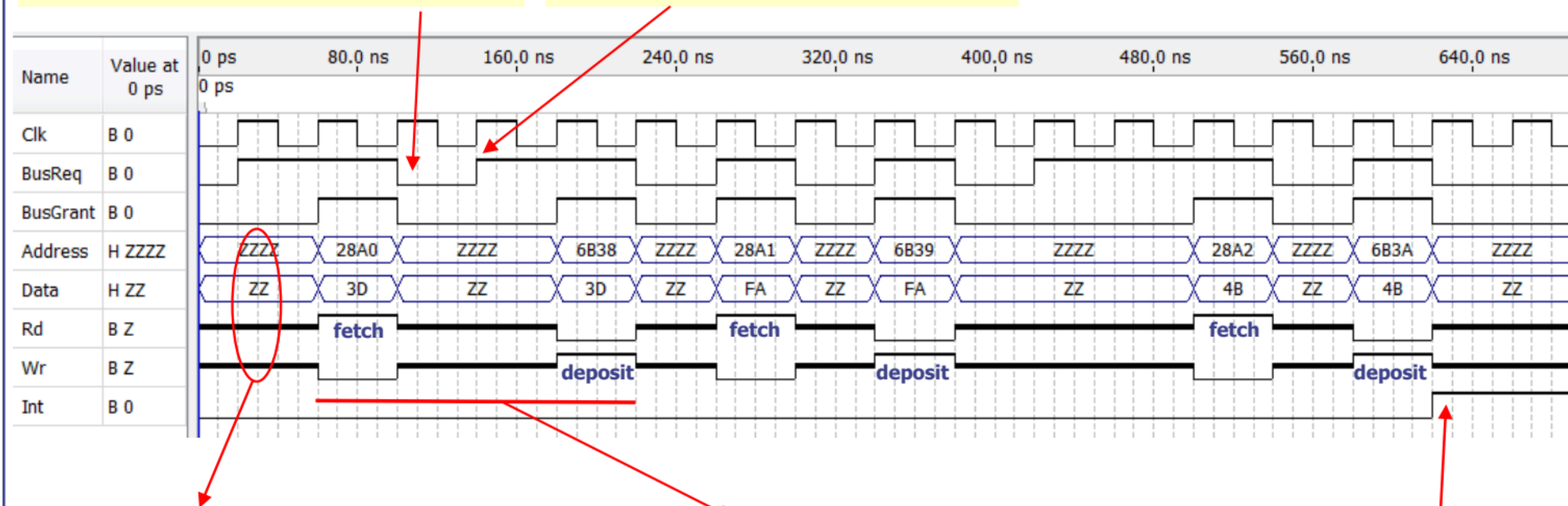
- Numa transferência em modo "cycle-stealing", o DMAC efetua a seguinte sequência de operações:
 - 1. Torna-se bus master**
 2. Fetch: lê 1 palavra do dispositivo fonte (do *source address*)
 - 3. Liberta os barramentos** - Deixa de ser bus master
 4. Espera durante um tempo fixo pré-determinado (Ex. 1T) - para dar CPU fazer oq precisar
 - 5. Torna-se bus master**
 6. Deposit: escreve 1 palavra no dispositivo destino (no *destination address*)
 - 7. Liberta os barramentos**
 8. Incrementa *source address* e *destination address*
 9. Incrementa o nº de bytes/words transferidos
 10. Espera durante um tempo fixo pré-determinado (Ex. 1T)
 11. Se não transferiu a totalidade do bloco, repete desde 1
 12. Após a transferência da totalidade dos dados, ativa o sinal de interrupção

Modo "Cycle-Stealing" (exemplo)

- Exemplo de uma transferência de 3 bytes (memória *byte-addressable*, barramento de dados de 8 bits, *bus cycle* = 1T, intervalo mínimo entre operações = 1T):
 - Endereço de início na origem: 0x28A0, [0x28A0, 0x28A2]
 - Endereço de início no destino: 0x6B38, [0x6B38, 0x6B3A]

DMAC desativa BusReq

DMAC reativa BusReq

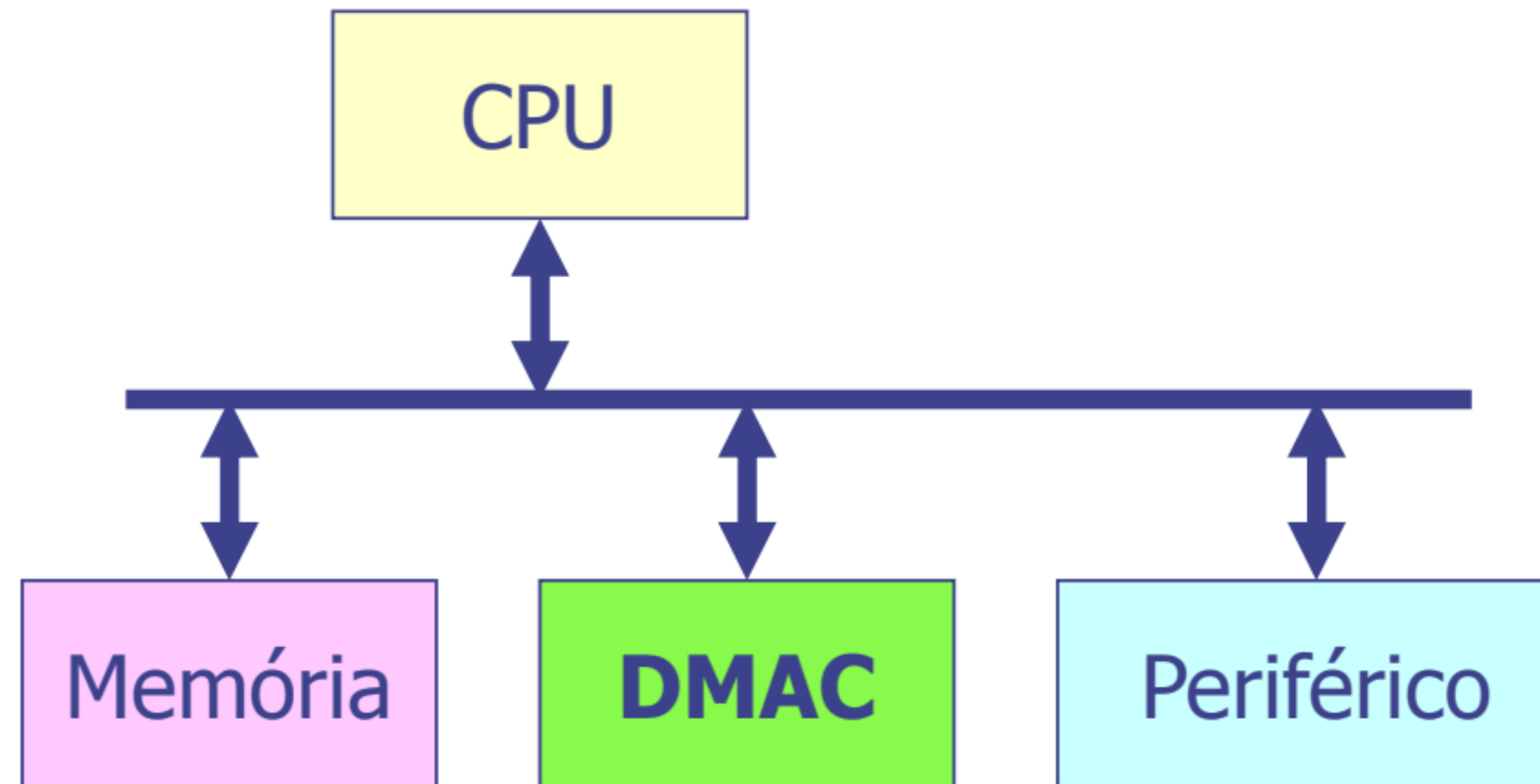


Barramentos do DMAC em alta impedância

Transferência do 1º byte

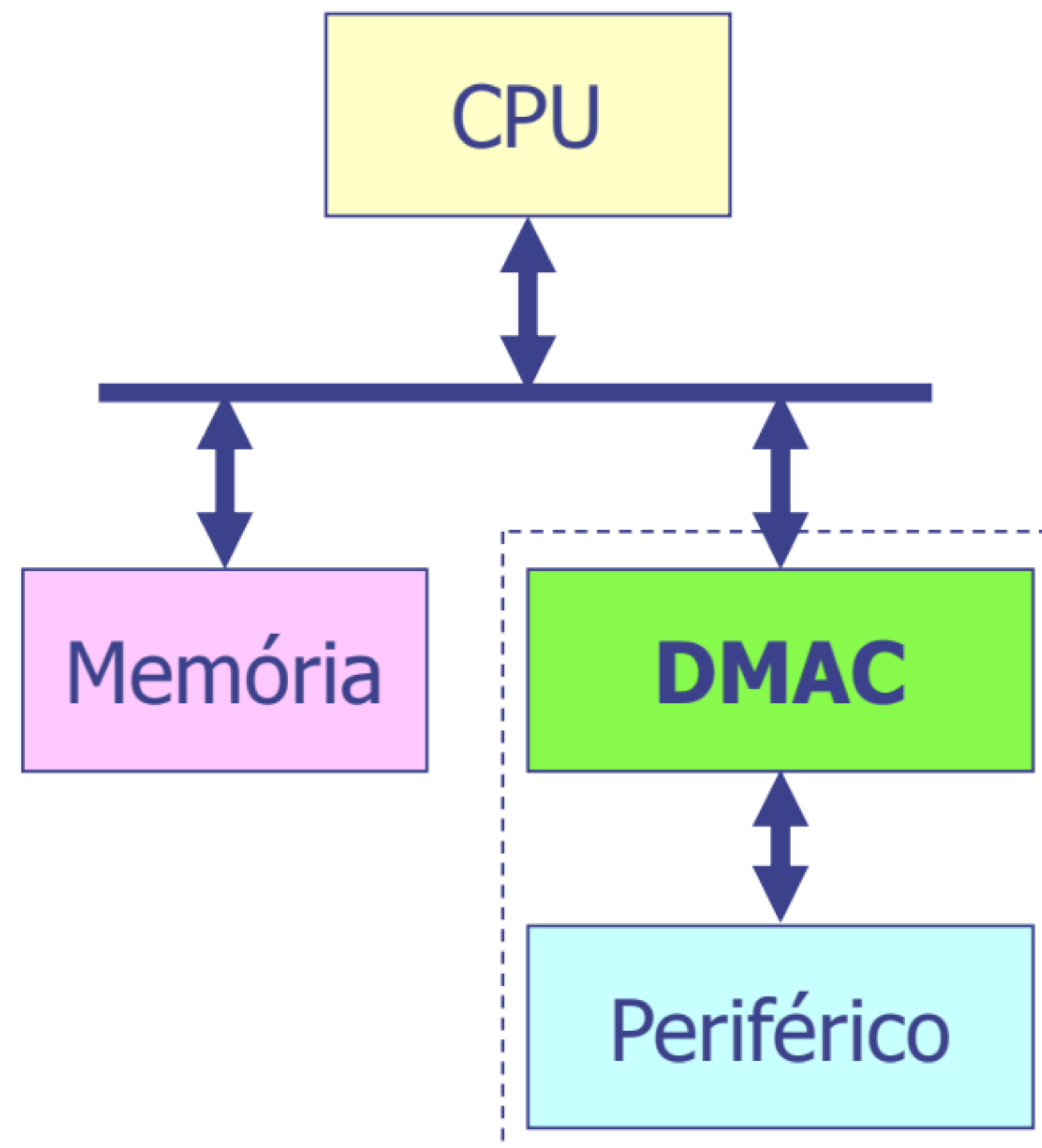
DMAC gera interrupção no fim da transferência

Configurações – Canal de DMA



- O DMAC fornece um serviço de transferência genérico
- Pode ser usado para transferir informação:
 - de I/O para memória
 - de memória para I/O
 - de memória para memória
- Para a transferência de 1 palavra o barramento é usado 2 vezes (2 "bus cycles" por palavra)
- Em modo "cycle stealing", por cada palavra transferida, o CPU liberta os barramentos 2 vezes

Configurações – DMA dedicado *- PIC 32 usa*



- O periférico tem o seu próprio controlador de DMA (**DMAC dedicado**) *cada periférico tem o seu (em vez de um DMA genérico)*
- Para a transferência completa de 1 palavra o barramento é usado apenas 1 vez (1 "bus cycle" por palavra): *→ para fetch ou depósito*
 - DMAC → memória ou memória → DMAC